

Seminar 6

The lecture gave you `map`, `filter`, `flatMap`, the folds and a few library combinators. Today we use them on problems the lecture did not show.

Rounds 1 and 2 drill the moves the homework needs. Rounds 3 and 4 are new: **folds that carry state** and **combinators that return `option`**.

You will find both, under other names, in Kotlin, Python, JS and C++.

Round 1 – a list against its own tail

```
def adjacentPairs (xs : List  $\alpha$ ) : List ( $\alpha \times \alpha$ ) :=  
  xs.zip xs.tail
```

```
def countRises (xs : List Int) : Nat :=  
  (adjacentPairs xs).countP (fun (x, y) => x < y)
```

lining the list up with itself

```
xs      = [1, 3, 2, 5]  
xs.tail = [3, 2, 5]  
zip     = [(1,3), (3,2), (2,5)]  
rises   =   ✓       .       ✓       -- 2
```

`zip` stops at the shorter list, so the last element pairs with nothing, and `[]` and `[x]` need no special case.

runningMax, ★ localMaxima

```
def runningMax : List Nat → List Nat
| [] => []
| x :: xs => xs.scanl max x
```

```
def localMaxima (xs : List Int) : List Int :=
  (List.zip xs (List.zip xs.tail xs.tail.tail)).filterMap
  fun (a, b, c) => if a < b ∧ c < b then some b else none
```

`scanl` keeps its starting value

```
[1, 4, 1, 5].scanl max 3
= [3, 3, 4, 4, 5]    -- one longer than the input
```

Start from the head: the output is exactly as long as the input.

Round 2 – lists from ranges

```
def timesTable (n : Nat) : List (List Nat) :=
  (List.range n).map fun i =>
    (List.range n).map fun j => (i + 1) * (j + 1)

def divisors (n : Nat) : List Nat :=
  (List.range (n + 1)).filter fun d => 0 < d && n % d == 0

def perfectBelow (n : Nat) : List Nat :=
  (List.range n).filter fun m =>
    0 < m && (divisors m).sum == 2 * m
```

the shapes

```
-- map inside map   ↪ a table
-- filter on range  ↪ "all k < n with ..."
-- divisors 6 = [1, 2, 3, 6], sum 12 = 2 * 6
```

`List.range n` starts at 0. Shift with `+ 1` or filter out 0. Do not hope that 0 behaves.

★ `twoSquares` – nested choices with `flatMap`

```
def twoSquares (n : Nat) : List (Nat × Nat) :=  
  (List.range (n + 1)).flatMap fun a =>  
    ((List.range (n + 1)).filter fun b =>  
      1 ≤ a && a ≤ b && a * a + b * b == n)  
    |>.map fun b => (a, b)
```

why `flatMap`, not `map`

```
-- each a gives a list of pairs  
-- map      : List (List (Nat × Nat))  
-- flatMap  : List (Nat × Nat)
```

One `flatMap` per "for each", one `filter` per condition; output order follows the loops.

Round 3 (new) – a fold is a loop

```
def stats (xs : List Nat) : Nat × Nat × Nat :=  
  xs.foldl (fun (count, total, top) x =>  
    (count + 1, total + x, max top x)) (0, 0, 0)
```

the loop it replaces (Kotlin)

```
var count = 0; var total = 0; var top = 0  
for (x in xs) {  
  count += 1; total += x; top = maxOf(top, x)  
}  
// variables   ↪ one tuple: the state  
// loop body   ↪ step : state → x → state
```

Several results from one pass: one tuple accumulator, not several folds.

balanced – a state that can break

```
def balanced (cs : List Char) : Bool :=
  let step : Option Nat → Char → Option Nat
    | some d, '(' => some (d + 1)
    | some (d + 1), ')' => some d
    | some 0, ')' => none
    | depth, _ => depth
  cs.foldl step (some 0) == some 0
```

`" ) ( "` step by step

```
some 0 -> ')' -> none -> '(' -> none
-- none never recovers: false
```

A bare depth counter accepts `" ) ( "`. The state needs an error value that never recovers.

dedupAdjacent, balances – what the state looks like

```
def dedupAdjacent [DecidableEq α] (xs : List α) : List α :=
  xs.foldr
    (fun x acc =>
      match acc with
      | y :: _ => if x = y then acc else x :: acc
      | [] => [x])
    []

def balances (start : Int) (moves : List Int) : List Int :=
  moves.scanl (· + ·) start
```

foldr sees the processed rest

```
1 :: [1, 2]    -- 1 = 1, keep acc
2 :: [3]       -- new run, cons
```

foldr lets the step **match** on what is already built. **scanl** is **foldl** that keeps every state it passed through.

foldl_add_start – the start must vary

```
theorem foldl_add_start (a : Nat) (xs : List Nat) :  
  xs.foldl (· + ·) a = a + xs.sum := by  
  induction xs generalizing a with  
  | nil => simp  
  | cons x xs ih => simp [ih]; omega
```

the **CONS** goal

```
ih : ∀ (a : ℕ), List.foldl (fun x1 x2 => x1 + x2) a xs  
    = a + xs.sum  
⊢ List.foldl (fun x1 x2 => x1 + x2) a (x :: xs)  
    = a + (x :: xs).sum
```

A **foldl** step changes the start to **a + x**, so the IH must hold for every start.

Round 4 (new) — search, don't loop

Lean	returns	Kotlin	Python
<code>xs.find? p</code>	first match, or <code>none</code>	<code>firstOrNull(p)</code>	<code>next((x for x in xs if p(x)), None)</code>
<code>xs.findIdx? p</code>	its position	<code>indexOfFirst(p)</code> <code>(-1)</code>	<code>next((i for i, x in enumerate(xs) if p(x)), None)</code>
<code>xs.filterMap f</code>	the <code>some</code> results	<code>mapNotNull(f)</code>	<code>[y for x in xs if (y := f(x)) is not None]</code>
<code>xs.lookup k</code>	value of the first <code>(k, v)</code>	<code>firstOrNull { it.first == k }?.second</code>	<code>next((v for k2, v in xs if k2 == k), None)</code>
<code>xs.takeWhile p</code> , <code>dropWhile p</code>	prefix / the rest	same names	<code>itertools.takewhile</code> , <code>dropwhile</code>
<code>xs.span p</code>	both at once	—	—
<code>xs.partition p</code>	(pass, fail)	<code>partition(p)</code>	two comprehensions
<code>xs.splitBy r</code>	runs of related neighbours	—	<code>itertools.groupby</code>
<code>xs.head?</code> , <code>xs.max?</code>	<code>none</code> on <code>[]</code>	<code>firstOrNull()</code> , <code>maxOrNull()</code>	<code>max(xs, default=None)</code>
<code>o.getD d</code>	unwrap with a default	<code>?: d</code>	<code>o if o is not None else d</code>

Instead of `-1` or an exception, the type says "might be missing", and the caller has to handle that case.

digitsIn, callerName

```
def digitsIn (cs : List Char) : List Nat :=
  cs.filterMap fun c =>
    if c.isDigit then some (c.toNat - '0'.toNat) else none

def callerName (book : List (String × String))
  (number : String) : String :=
  (book.lookup number).getD "unknown"
```

one pass where you would write two

```
-- filter isDigit, then map value
--   = filterMap (one function decides both)
-- lookup gives the FIRST entry: "Ada", not "Grace"
```

filterMap fuses "keep it?" and "turn it into what?" into one function, which suits a parse that can fail per element.

splitPassFail, firstOverdraft?

```
def splitPassFail (threshold : Nat)
  (results : List (String × Nat)) : List String × List String :=
  let (pass, fail) := results.partition fun r => threshold ≤ r.2
  (pass.map (·.1), fail.map (·.1))

def firstOverdraft? (start : Int) (moves : List Int) : Option Nat :=
  (balances start moves).findIdx? (· < 0)
```

reusing round 3

```
balances 10 [-3, 5, -20, 30, -40]
  = [10, 7, 12, -8, 22, -18]
  --           ↑ index 3
```

Build the history with `scanl`, then search it with `findIdx?`.

`find?_eq_head?_filter`, ★ `takeWhile_append_dropWhile`

```
theorem find?_eq_head?_filter (p :  $\alpha \rightarrow \text{Bool}$ ) (xs : List  $\alpha$ ) :  
  xs.find? p = (xs.filter p).head? := by  
  induction xs with  
  | nil => rfl  
  | cons x xs ih =>  
    cases h : p x <;> simp only [List.find?, List.filter, h, ih, List.head?_cons]  
  
theorem takeWhile_append_dropWhile (p :  $\alpha \rightarrow \text{Bool}$ ) (xs : List  $\alpha$ ) :  
  takeWhile p xs ++ dropWhile p xs = xs := by  
  induction xs with  
  | nil => rfl  
  | cons x xs ih => cases h : p x <;> simp [takeWhile, dropWhile, h, ih]
```

Every definition here branches on `p x`, so the proof branches on it too: `cases h : p x` keeps the equation `h` that lets `simp` pick the right branch.

The same vocabulary elsewhere

Lean	Kotlin	Python	JS	C++
<code>map f</code>	<code>map</code>	<code>map</code> , comprehension	<code>map</code>	<code>std::views::transform</code>
<code>filter p</code>	<code>filter</code>	<code>filter</code> , comprehension	<code>filter</code>	<code>std::views::filter</code>
<code>flatMap f</code>	<code>flatMap</code>	nested comprehension	<code>flatMap</code>	<code>views::transform </code> <code>views::join</code>
<code>foldl f z</code>	<code>fold(z, f)</code>	<code>functools.reduce(f,</code> <code>xs, z)</code>	<code>reduce(f, z)</code>	<code>std::accumulate</code>
<code>foldr f z</code>	<code>foldRight(z, f)</code>	—	<code>reduceRight</code>	<code>std::accumulate</code> on reversed range
<code>scanl f z</code>	<code>runningFold(z, f)</code>	<code>itertools.accumulate</code>	—	<code>std::inclusive_scan</code>
<code>zip</code> , <code>zipWith</code>	<code>zip</code> , <code>zip { }</code>	<code>zip</code>	—	<code>std::views::zip</code> (C++23)
<code>find?</code> , <code>filterMap</code>	<code>firstOrNull</code> , <code>mapNotNull</code>	<code>next(..., None)</code> , comprehension	<code>find</code> (undefined)	<code>std::ranges::find_if</code>
<code>takeWhile</code> , <code>partition</code>	same names	<code>itertools.takewhile</code> , —	—	<code>views::take_while</code> , <code>std::partition_copy</code>